

How to automate result reporting

September 14, 2026

<https://github.com/jimb0w/automation>

Jedidiah Morton

Jedidiah.Morton@monash.edu

Research Fellow

Baker Heart and Diabetes Institute, Melbourne, Australia

Monash University, Melbourne, Australia

Contents

1	Data cleaning	2
2	Analysis	7

1 Data cleaning

Document everything! Say what you're going to do, do it, then comment on it.

For example: I'm going to create a random health dataset with the following variables:

- Sex (0 = female; 1 = male)
- Age (in years)
- Diabetes status (0 = no diabetes; 1 = diabetes)
- Smoking status (0 = non-smoker; 1 = smoker)
- LDL-C (in mmol/L)
- Outcome (in outcome units)

As so:

```
cd /home/jimb0w/Downloads/tempauto/  
clear  
set obs 20000  
set seed 1312  
gen sex = runiformint(0,1)  
gen age = rnormal(60,15)  
gen diabetes = runiformint(0,1)  
gen smoking = runiformint(0,1)  
gen ldl = rnormal(3,0.5)  
gen outcome = rnormal(200,30)+diabetes*rnormal(100,20)  
export delimited using randomhealthdata.csv, replace  
save randomhealthdata, replace
```

Now I'm going to check the variables make sense:

```
. su(sex), detail
```

sex				
	Percentiles	Smallest		
1%	0	0		
5%	0	0		
10%	0	0	Obs	20,000
25%	0	0	Sum of wgt.	20,000
50%	0		Mean	.4989
		Largest	Std. dev.	.5000113
75%	1	1		
90%	1	1	Variance	.2500113
95%	1	1	Skewness	.0044
99%	1	1	Kurtosis	1.000019

```
. su(diab), detail
```

diabetes				
	Percentiles	Smallest		
1%	0	0		
5%	0	0		
10%	0	0	Obs	20,000
25%	0	0	Sum of wgt.	20,000
50%	0		Mean	.494
		Largest	Std. dev.	.4999765
75%	1	1		
90%	1	1	Variance	.2499765
95%	1	1	Skewness	.0240017
99%	1	1	Kurtosis	1.000576

```
. su(smoking), detail
```

smoking				
	Percentiles	Smallest		
1%	0	0		
5%	0	0		
10%	0	0	Obs	20,000
25%	0	0	Sum of wgt.	20,000
50%	1		Mean	.50085
		Largest	Std. dev.	.5000118
75%	1	1		
90%	1	1	Variance	.2500118
95%	1	1	Skewness	-.0034
99%	1	1	Kurtosis	1.000012

```
. su(age), detail
```

age				
	Percentiles	Smallest		
1%	24.66596	.835979		
5%	35.15182	1.354185		
10%	40.73132	4.960024	Obs	20,000
25%	49.76038	5.434331	Sum of wgt.	20,000
50%	59.77124		Mean	59.82388
		Largest	Std. dev.	14.99574
75%	69.96964	109.9906		
90%	78.95889	111.0537	Variance	224.8723
95%	84.55445	111.059	Skewness	-.0096071
99%	94.6009	115.5846	Kurtosis	3.032043

```
. su(ldl), detail
```

ldl				
	Percentiles	Smallest		
1%	1.81618	1.169115		
5%	2.166875	1.17411		
10%	2.354322	1.184871	Obs	20,000
25%	2.66082	1.20128	Sum of wgt.	20,000
50%	3.000187		Mean	2.997539
		Largest	Std. dev.	.50188
75%	3.334033	4.78299		
90%	3.636441	4.849288	Variance	.2518836
95%	3.817179	4.876576	Skewness	-.0133279
99%	4.168352	5.034541	Kurtosis	3.031805

```
. su(outcome), detail
```

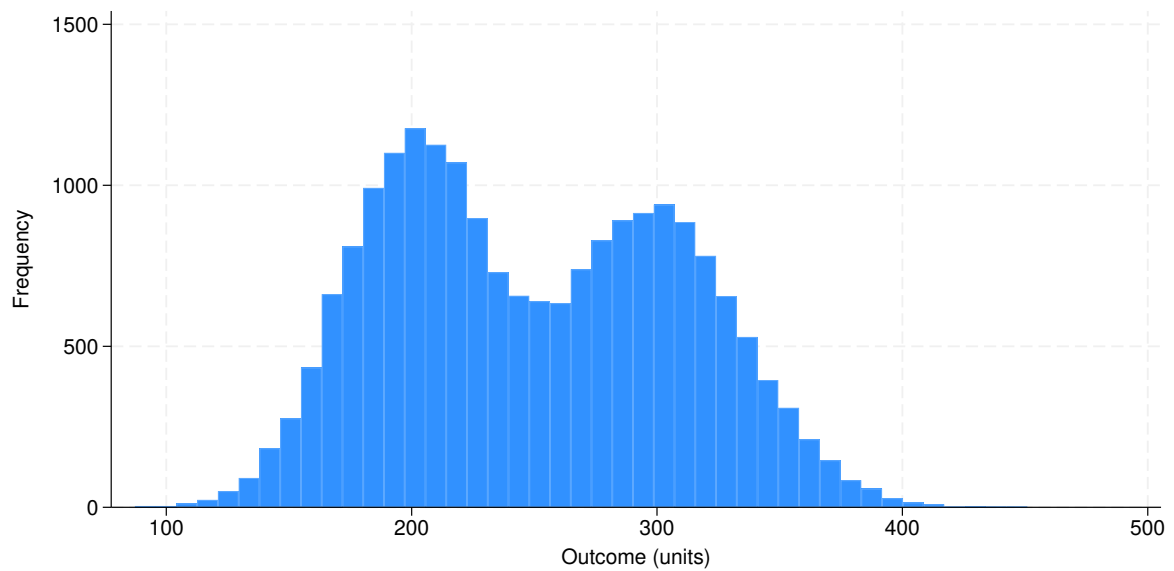
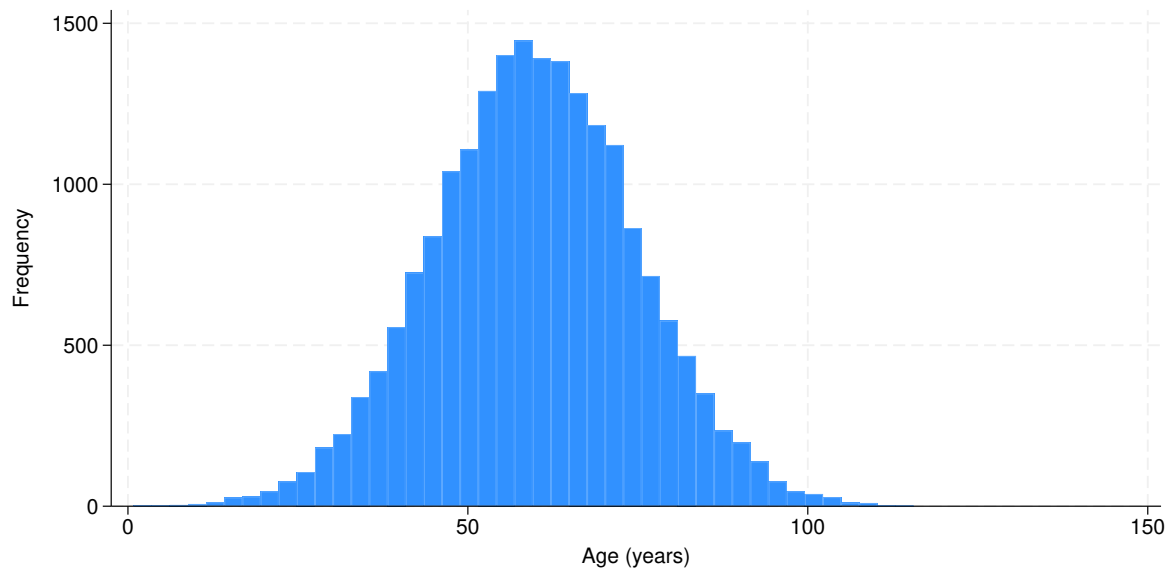
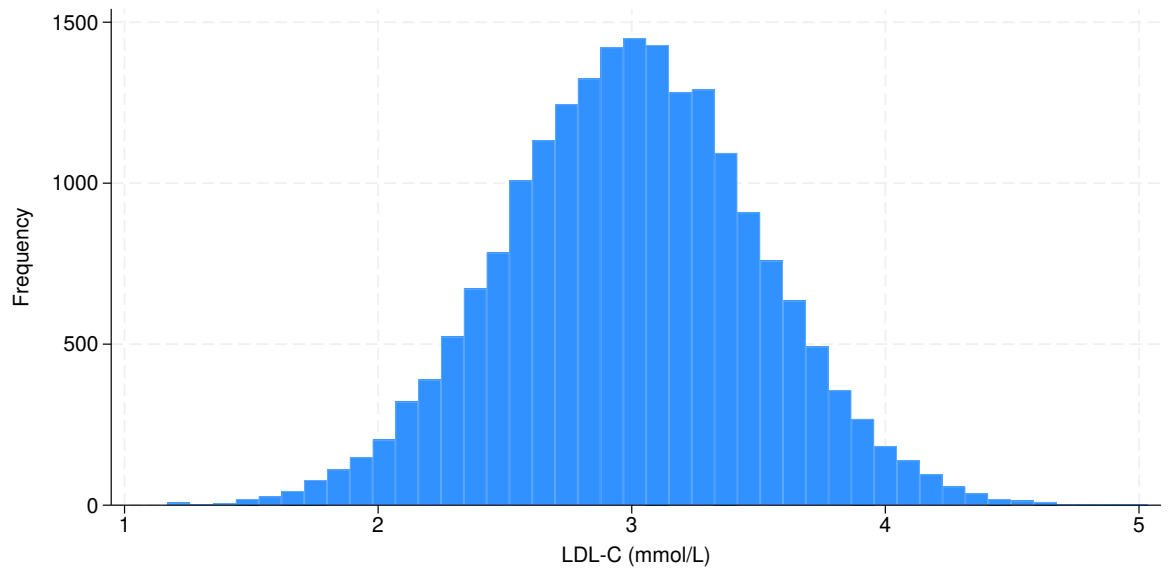
outcome				
	Percentiles	Smallest		
1%	139.2732	87.21138		
5%	162.0865	91.83277		
10%	174.7943	91.93818	Obs	20,000
25%	199.8294	92.32668	Sum of wgt.	20,000
50%	244.0778		Mean	249.3994
		Largest	Std. dev.	59.80981
75%	299.1514	426.0753		
90%	329.6298	430.0988	Variance	3577.214
95%	345.9959	434.5064	Skewness	.163877
99%	374.6187	450.9283	Kurtosis	2.073516

It looks like these variables make sense and there don't appear to be any errors with the data. But we should check further: let's produce a histogram to check the continuous variables. The code below produces Figure 1.1.

```
mkdir GPH
```

```
hist ldl, frequency ///
xtitle("LDL-C (mmol/L)")
graph save GPH/LDL_hist, replace
hist age, frequency ///
xtitle("Age (years)")
graph save GPH/Age_hist, replace
hist outcome, frequency ///
xtitle("Outcome (units)")
graph save GPH/Outcome_hist, replace
graph combine ///
GPH/LDL_hist.gph ///
GPH/Age_hist.gph ///
GPH/Outcome_hist.gph ///
, cols(1) altshrink xsize(3)
graph export GPH/distcheck.pdf, as(pdf) replace
```

Figure 1.1: Histograms of LDL-C, Age, and outcome.



It appears that our age and LDL-C variables are normally distributed and seem reasonable. there is an odd, bimodal distribution in the outcome variable.

In terms of automation, the key thing here was producing my Figures using code, rather than a graphics program, and automatically exporting them to pdf, before importing them and displaying them within the document. Notice also that Figure 1.1 hyperlinks to the figure if referenced properly, so if you update the figure order later, you don't have to re-number all your figure references in the text. That means if you change anything in your analysis, add a figure, etc., you don't need to re-make the figure, just re-run the code and it will re-appear updated in this pdf file.

2 Analysis

Before beginning our analysis, we should present a summary table of population characteristics. We are most interested in diabetes, so we intend to stratify this table by diabetes type.

Here, we are going to collect the results from the Stata output and store them in a matrix, before saving that matrix to the Stata dataset editor and formatting the outputs for presentation in a Table. It helps to first think about the Table you want to present before collecting the results. The Table 2.1 is what we're aiming for.

Table 2.1: Table we are aiming for.

Outcome	Overall	Diabetes status	
		No diabetes	Diabetes
Sex	N (%)	N (%)	N (%)
Smoking status	N (%)	N (%)	N (%)
Age	median (IQR)	median (IQR)	median (IQR)
LDL-C	median (IQR)	median (IQR)	median (IQR)
Outcome	median (IQR)	median (IQR)	median (IQR)

Data are presented as N (%) or median (IQR).

Now that we know what we are aiming for, we need to collect the values. Stata's *return list* command is useful here: it tells you what is stored in Stata's memory, and thus how we can access it. For example:

```
. use randomhealthdata, clear
. su(ldl), detail
```

ldl				
	Percentiles	Smallest		
1%	1.81618	1.169115		
5%	2.166875	1.17411		
10%	2.354322	1.184871	Obs	20,000
25%	2.66082	1.20128	Sum of wgt.	20,000
50%	3.000187		Mean	2.997539
		Largest	Std. dev.	.50188
75%	3.334033	4.78299		
90%	3.636441	4.849288	Variance	.2518836
95%	3.817179	4.876576	Skewness	-.0133279
99%	4.168352	5.034541	Kurtosis	3.031805

```
. return list
scalars:
      r(N) = 20000
    r(sum_w) = 20000
    r(mean) = 2.99753932904005
    r(Var) = .2518835571674297
    r(sd) = .5018800226821443
  r(skewness) = -.0133279276969729
  r(kurtosis) = 3.031805267881794
    r(sum) = 59950.78658080101
    r(min) = 1.169115424156189
    r(max) = 5.03454065322876
    r(p1) = 1.816179573535919
    r(p5) = 2.166874527931213
    r(p10) = 2.354321837425232
    r(p25) = 2.660820245742798
    r(p50) = 3.000186562538147
    r(p75) = 3.334033250808716
```



```
r(p90) = 3.636441469192505  
r(p95) = 3.817179083824158  
r(p99) = 4.168352127075195
```

We can access any one of those statistics and store it for use in our table. (Annoyingly “`tabulate`” doesn’t store anything useful, so you have to use the *matcell* option.)

So, let’s get all the data we need for our table: